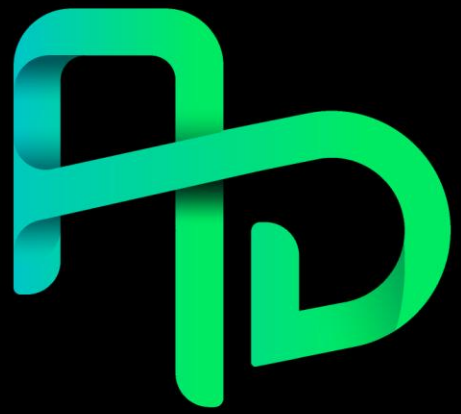


ADVANCED TECHNOLOGY DAYS

3–4 NOVEMBER 2025
MOZAIK EVENT CENTER, ZAGREB

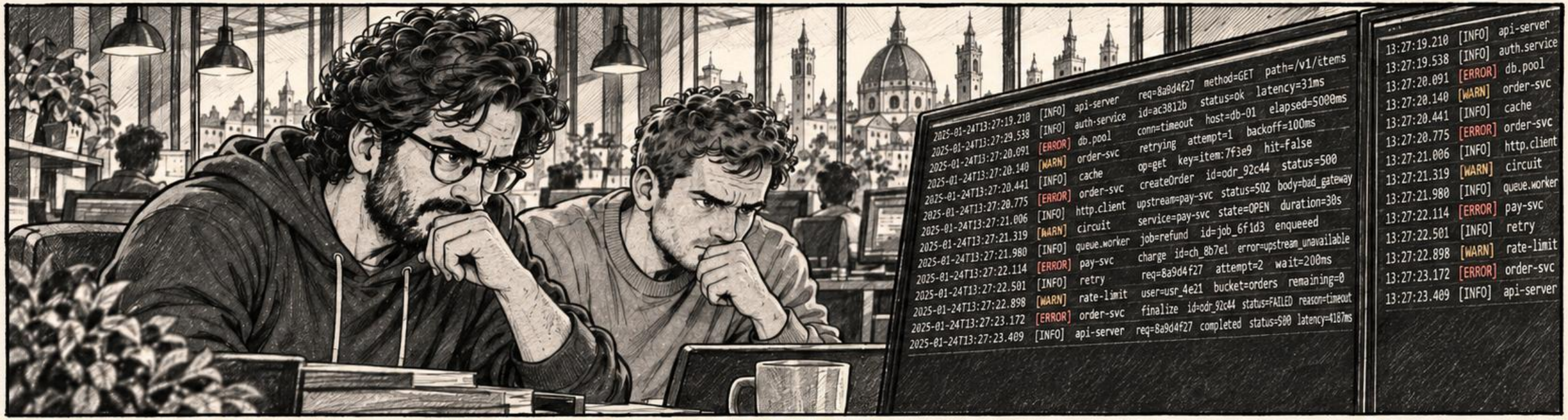
ADVANCED TECHNOLOGY DAYS

Observability in action with OpenTelemetry and .NET

Alessandro Mengoli
Technical Leader @ SACMI








```
2025-01-24T13:27:19.872  [INFO]   api-gateway   trace=9d1e2b3c7f  span=1/4      svc=api-gw     req=8a9d4f27  method=GET   path=/v1/orders/odr_92c44  status=200  latency=24ms
2025-01-24T13:27:20.091  [ERROR]  db.pool       svc=db            id=conn_7a2f  host=db-01     event=acquire  error=timeout               timeout=5000ms  retries=0
2025-01-24T13:27:20.140  [WARN]   order-svc     svc=order         id=odr_92c44  msg=retrying request          attempt=1  backoff=100ms  cause=db_timeout
2025-01-24T13:27:20.441  [INFO]   cache         svc=cache         key=item:7f3e9  op=GET               hit=false  ttl=300s  elapsed=3ms
2025-01-24T13:27:20.775  [ERROR]  order-svc     svc=order         id=odr_92c44  op=createOrder       status=500  error=InternalError  code=E1003  msg="failed to persist order"
2025-01-24T13:27:21.006  [INFO]   http.client   svc=order         upstream=pay-svc  req=ch_8b7e1     status=502  error=bad_gateway  latency=721ms
2025-01-24T13:27:21.319  [WARN]   circuit       svc=pay-svc       state=OPEN    since=13:27:21.319  duration=30s  failures=5  threshold=5
2025-01-24T13:27:21.980  [INFO]   queue.worker  svc=worker        job=refund    id=job_6f1d3     status=enqueued  queue=refunds  size=427
2025-01-24T13:27:22.114  [ERROR]  pay-svc       svc=pay-svc       id=ch_8b7e1   op=charge        status=FAILED  error=upstream_unavailable  code=E502  retryable=true
2025-01-24T13:27:22.501  [INFO]   retry         svc=order         req=8a9d4f27  attempt=2        wait=200ms  policy=exponential  max=3
2025-01-24T13:27:22.508  [WARN]   rate-limit    svc=api-gw        user=usr_4e21  bucket=orders   remaining=0  limit=100/min  action=throttle
2025-01-24T13:27:23.172  [ERROR]  order-svc     svc=order         id=odr_92c44  op=finalize      status=FAILED  reason=timeout  elapsed=4182ms
2025-01-24T13:27:23.409  [INFO]   api-server    svc=api           req=8a9d4f27  completed        status=500  latency=4187ms  trace=9d1e2b3c7f
```





A system is observable if, by knowing its outputs over a given period of time, it is possible to fully determine its internal state.

R. E. Kálmán (1960), “On the general theory of control systems”

Observability in software

01

How is it doing?

Understand the state and behaviour of the running application

02

No new code

Inspect the internal state without adding code or redeploying

03

Unknown unknowns

Answer questions and unexpected problems nobody anticipated

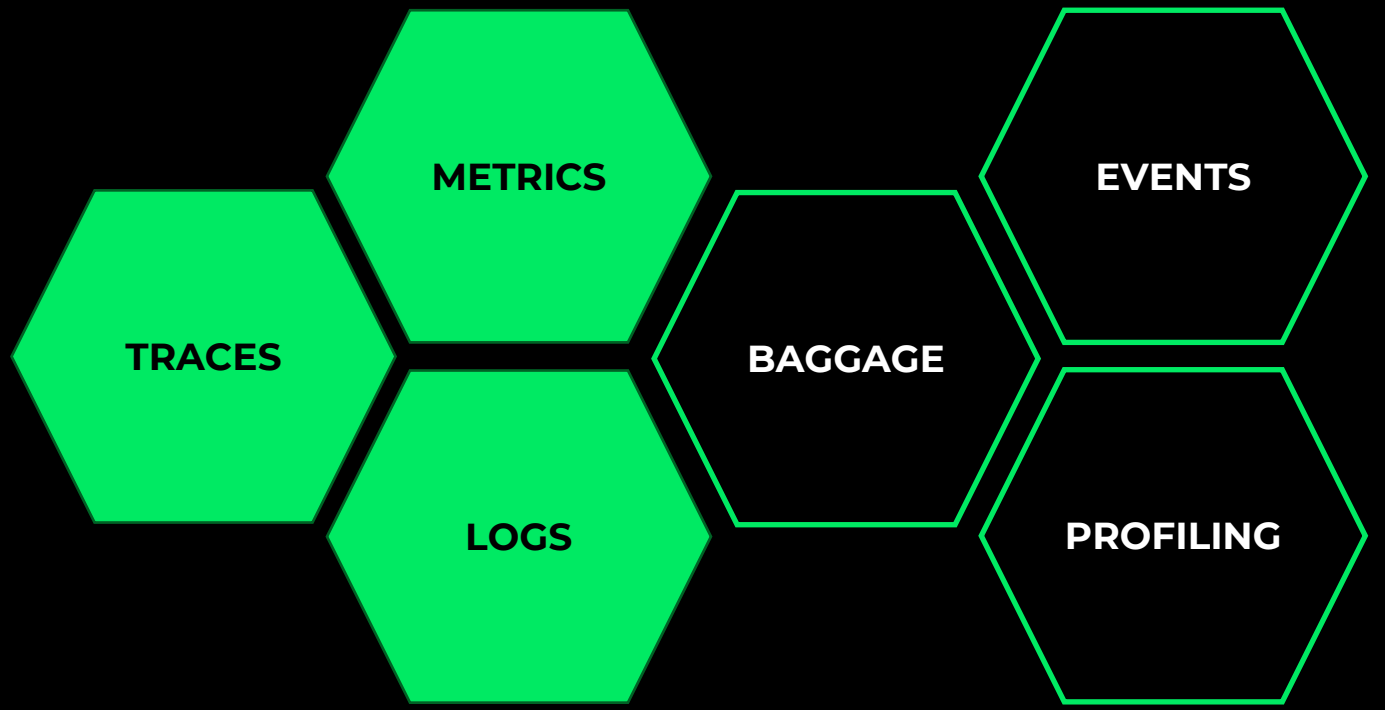
Observability != Monitoring

OpenTelemetry

- **CNCF project**, same ecosystem as Kubernetes and Prometheus
- **One standard** to collect telemetry and send it to any backend
- **Vendor neutral**, backed by all the major vendors
- **De facto standard** for observability
- **Many languages**: C++, C#, Go, Java, Python, ...



Signals



Metrics

Numeric representation of performance or state, aggregated over time

01

Counter

Cumulative data (e.g. number of HTTP requests)

02

Gauge

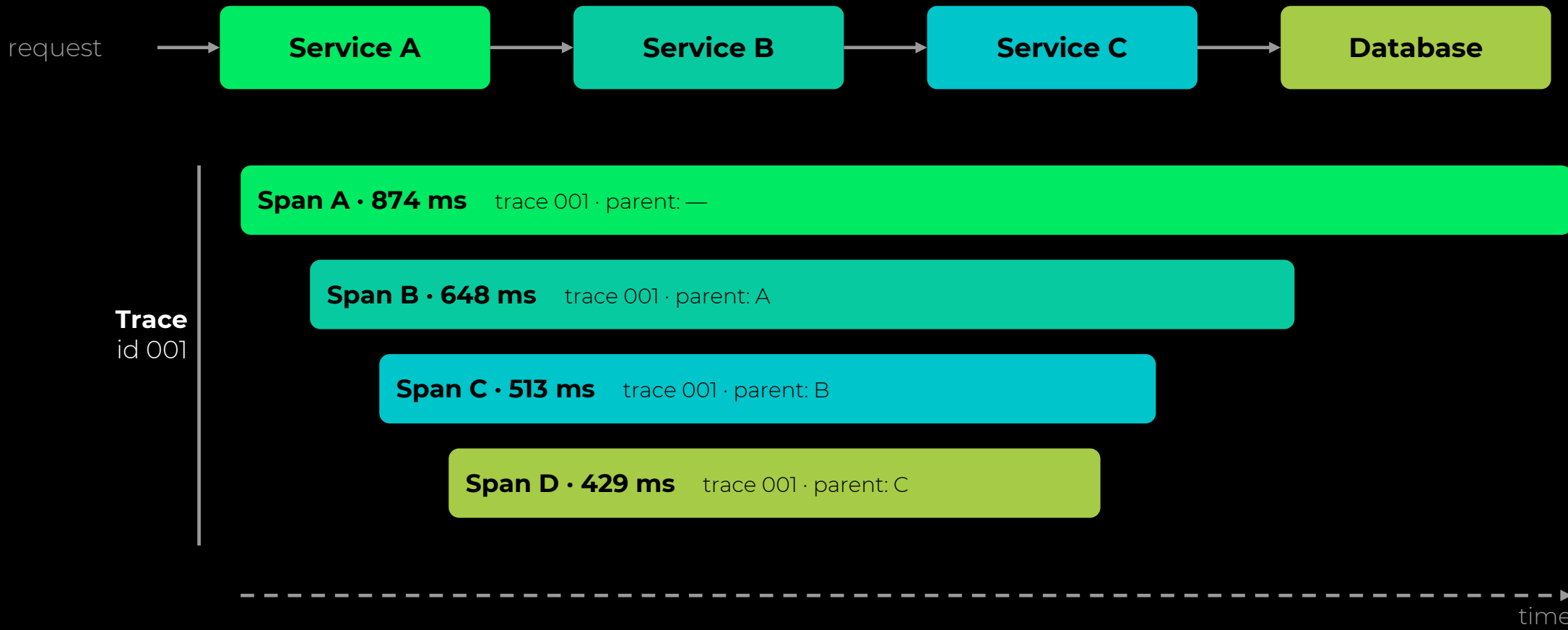
Current value, goes up and down (e.g. CPU usage)

03

Histogram

Distribution of values (e.g. request latency)

Traces


request
Service A
Service B
Service C
Database
Trace
id 001
Span A · 874 ms
trace 001 · parent: —
Span B · 648 ms
trace 001 · parent: A
Span C · 513 ms
trace 001 · parent: B
Span D · 429 ms
trace 001 · parent: C
time

Logs

Timestamped records enriched with context

Unstructured vs structured

✗

```
logger.LogDebug($"Created order with id {orderId}");
```

✓

```
logger.LogDebug("Created order with {OrderId}", orderId);
```

Unified context



Resource

The entity producing the telemetry

A set of attributes attached to every trace, metric and log (App name and version, environment, host, cluster, region, cloud provider, OS…)

```
service.name = "orders-api"
```

```
service.version = "1.4.2"
```

```
deployment.environment.name = "prod"
```

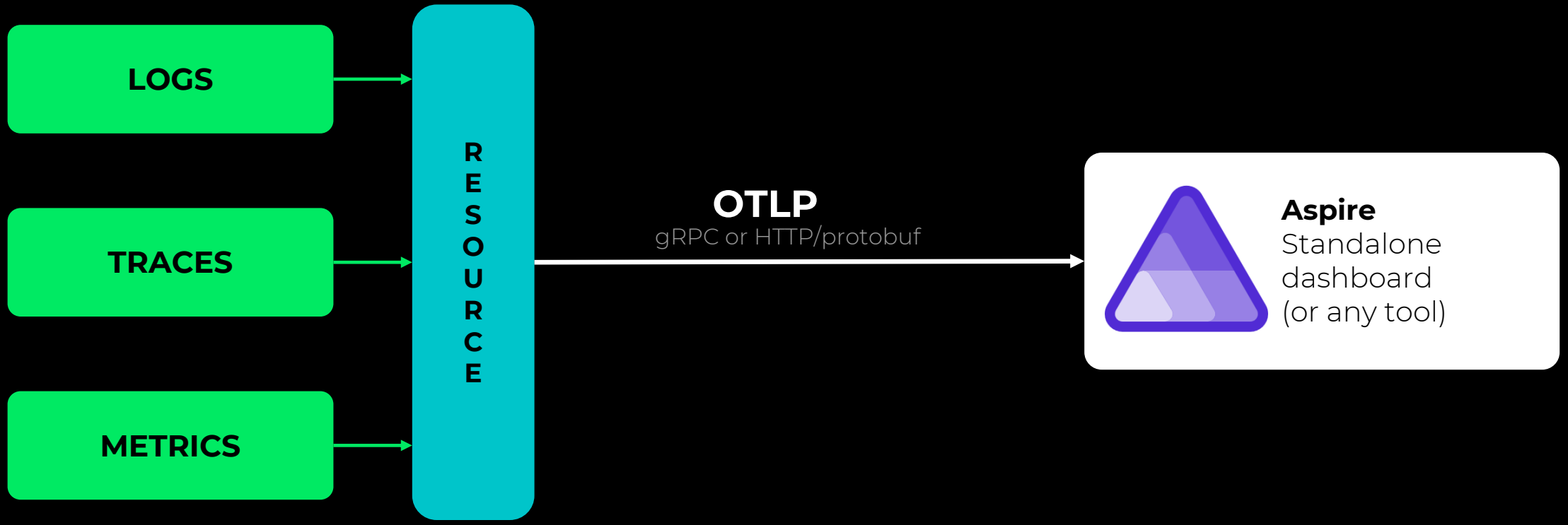
```
host.name = "vm-orders-01"
```

```
cloud.provider = "azure"
```

```
cloud.region = "westeurope"
```

```
os.type = "linux"
```

From code to backend



DEMO

ADVANCED
TECHNOLOGY
DAYS

Standalone dashboard

```
docker run --rm -it -d \
  -p 18888:18888 \
  -p 4317:18889 \
  --name aspire-dashboard \
  mcr.microsoft.com/dotnet/
  aspire-dashboard:latest
```

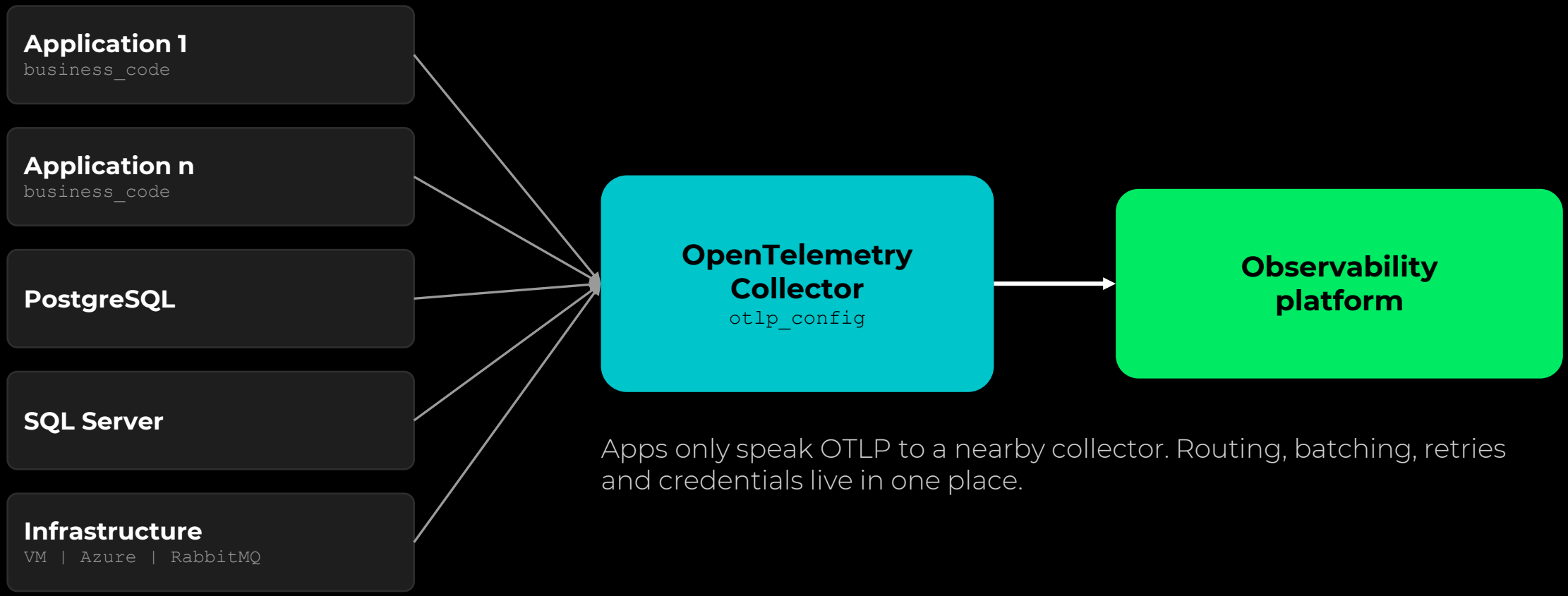
A few lines of code

```
builder.Services.AddOpenTelemetry()
    .ConfigureResource(r => r.AddService(
        serviceName: "example", serviceVersion: "1.0.0"))
    .WithTracing(tracing => tracing
        .AddAspNetCoreInstrumentation()
        .AddHttpClientInstrumentation()
        .AddOtlpExporter())
    .WithMetrics(metrics => metrics
        .AddAspNetCoreInstrumentation()
        .AddHttpClientInstrumentation()
        .AddOtlpExporter())
    .WithLogging(logging => logging.AddOtlpExporter());
```

DEMO

ADVANCED
TECHNOLOGY
DAYS

Enter the Collector



Apps only speak OTLP to a nearby collector. Routing, batching, retries and credentials live in one place.

OpenTelemetry Collector

Applications

Infrastructure

Receivers







• • •

Processors

Memory limiter

Sampling

Filtering

Batch

Attributes

Resource

Obfuscate

Exporters







• • •

Backends

Why it pays off



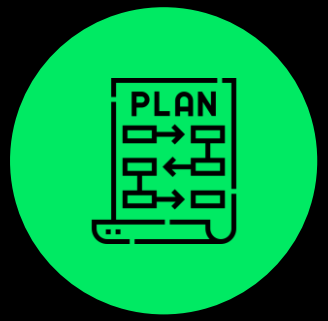
Troubleshooting

Find the root cause in minutes, not hours



Performance

Identify bottlenecks and see where time is spent



Product decisions

Data on how features are actually used

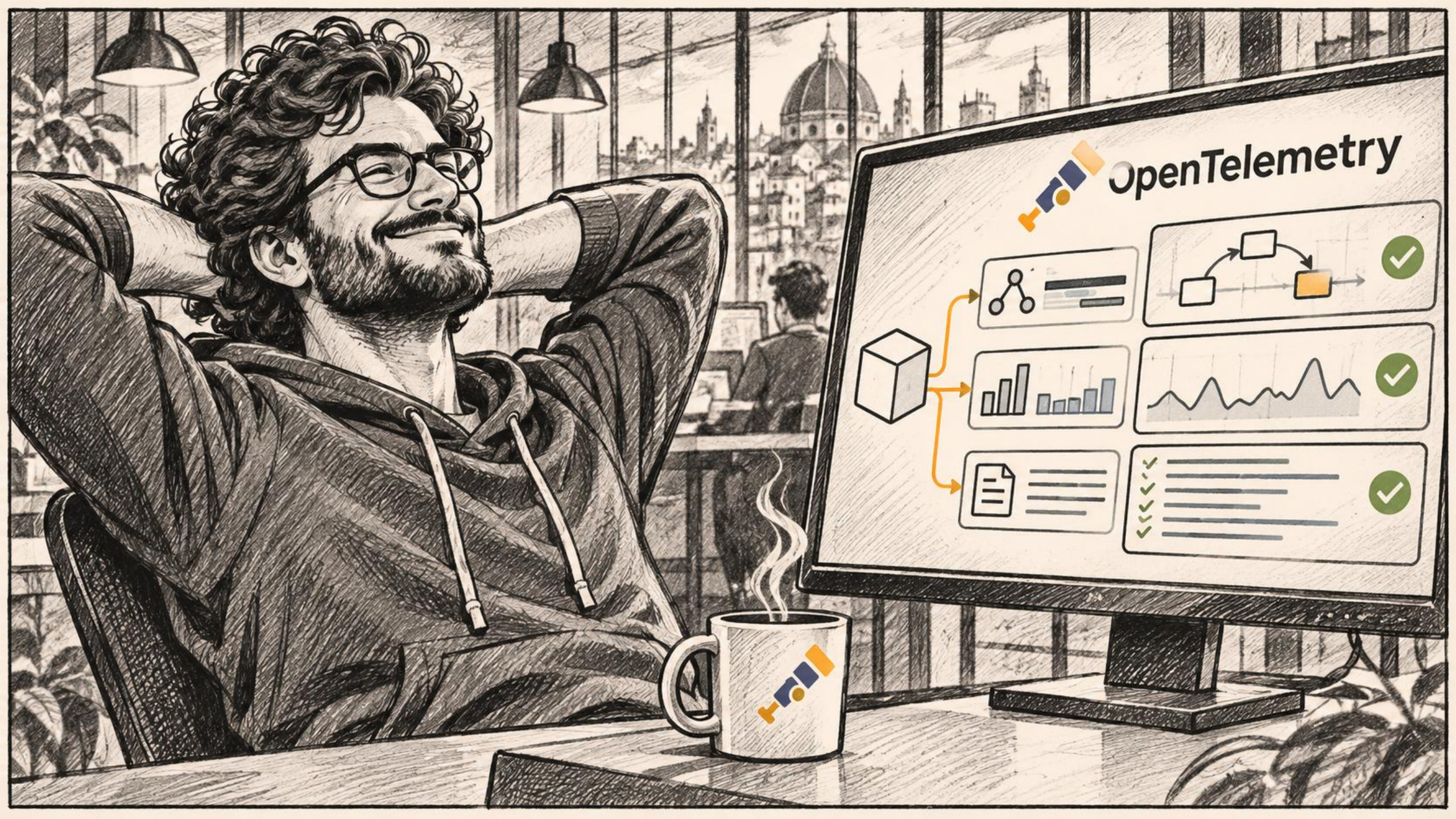


GenAI

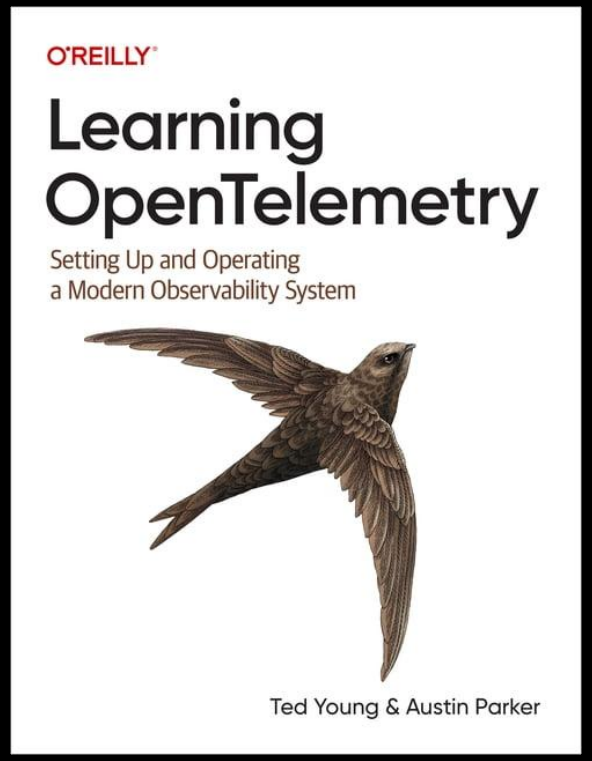
OpenTelemetry already provides GenAI semantic conventions

Takeaways

1. Observability lets you understand, diagnose and govern your systems
2. A few lines of code make a .NET app observable with OpenTelemetry, do it as soon as you can
3. Useful also for operations and product managers alike; so create the first PoC and share it
4. Make Joe a happy man!


OpenTelemetry

About & resources



Learning OpenTelemetry
T. Young, A. Parker · O'Reilly

Alessandro Mengoli

Technical Leader @ SACMI

LinkedIn	alessandro-mengoli
GitHub	amengoli9/talks
Blog	devpills.net
Community	devromagna.org

Feedback!

